Academic Year: 2026-2027
Semester: III
Class / Branch: SE IT
Subject: ADSAL LAB
Name of Instructor:prof.Sneha Dalvi

Name of Student: Nishant Suryarao.
Student ID: 26204001.
Date of Performance:13/10/2026
Date of Submission:13/10/2026

Experiment No.12

Aim: Implement Longest Common Subsequence (LCS) algorithm.

Code:

```
#include <stdio.h>
#include <string.h>

int max(int a, int b) {
return (a > b) ? a : b;
}

int lcs(char X[], char Y[]) {
int m = strlen(X); int n =
strlen(Y); int L[m + 1][n +
1];

for (int i = 0; i <= m; i++) { for
(int j = 0; j <= n; j++) {

if (i == 0 || j == 0) L[i][j]
= 0;

else if (X[i - 1] == Y[j - 1]) L[i][j] =
L[i - 1][j - 1] + 1;

else
L[i][j] = max(L[i - 1][j],
L[i][j - 1]);
```

```
}
}

return L[m][n];
}

int main() { char
X[100], Y[100];

printf("Enter first sequence: ");
scanf("%99s", X);

printf("Enter second sequence: ");

scanf("%99s", Y); printf("Length of LCS =

%d\n", lcs(X, Y));

return 0;
}
```

Output Screen Shots:

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/Desktop/Nishant$ gcc -o Exp12 Exp12.c

apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/Desktop/Nishant$ ./Exp12
Enter first sequence: ABABCD
Enter second sequence: BCDCA
Length of LCS = 3
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/Desktop/Nishant$
```

Conclusion:

The Longest Common Subsequence (LCS) algorithm was successfully implemented using Dynamic Programming. The program compares the two sequences and stores the results of smaller subproblems in a DP table to efficiently find the LCS length. For the given
example, "ABCBDAB" and "BDCABA", the length of the LCS is 4. This experiment helped in understanding how Dynamic Programming can be used to solve problems efficiently by avoiding repeated calculations.

Experiment 12: Longest Common Subsequence (LCS)

Definition: The Longest Common Subsequence problem finds the longest sequence of characters that appears in the same relative order in two given sequences, without requiring the characters to be consecutive.

Algorithm:

1. Start.
2. Read the two sequences, X and Y.
3. Find their lengths, \(m\) and \(n\).
4. Create a DP table of size \((m+1)\times(n+1)\).
5. Initialize the first row and column with zero.
6. If the current characters match, set L[i][j] = L[i-1][j-1] + 1.
7. Otherwise, set L[i][j] to the maximum of L[i-1][j] and L[i][j-1].
8. Display L[m][n] as the LCS length.
9. Stop.

Conclusion: Thus, the Longest Common Subsequence algorithm was successfully implemented using Dynamic Programming to determine the length of the common subsequence.

Complexity: Time \(O(mn)\), Space \(O(mn)\).